

Phishing Detection Using Machine Learning: Evaluating Classical Classifiers for Web-Based Phishing Detection

Keshav Gupta(2210990505), Muskan Saini(2210991945),
Aishwary Chauhan(2210991209), Tejasvi Jhamta(2210992464)
Department of Computer Science & Engineering, Chitkara University
Final Year B.E – Batch 2022–26

Abstract - Phishing continues to be a major headache in the cybersecurity space we explored how far classical, lightweight ML models could get against this problem without demanding heavy compute. Working on a 549,347 sample Kaggle dataset of labelled URLs we built a TF-IDF based pipeline and put two classifiers head to head : Logistic Regression (LR) as our main model and Multinomial Naive Bayes (MNB) as a comparison. Neither model ever loads the actual webpage classification is done entirely on the raw URL string no WHOIS queries no page rendering no outbound network calls. LR came out ahead with 96.4% accuracy and an AUC of 0.984 while MNB reached 94.1% accuracy and AUC 0.971. Both classifiers ran in sub millisecond time per URL and were deployed through a Flask web app for live testing. We report results via confusion matrices, ROC curves and learning curves.

Index Terms - Phishing Detection, URL Classification, Logistic Regression, Naive Bayes, TF-IDF, Machine Learning, Cybersecurity, Flask.

I. INTRODUCTION

Phishing has been around for decades, and if anything, it has grown more dangerous rather than less. The core trick hasn't shifted much dupe someone into clicking a spoofed link and surrendering credentials or payment details but the volume has grown dramatically. APWG data shows the number of detected phishing sites jumped from roughly 266,000 in 2019 to well over 1.6 million by 2024 [13]. Financial damage tracked a similar trajectory climbing from around USD 1.8 billion in 2019 to an estimated USD 24.7 billion by 2024 a staggering increase.

The standard countermeasure is a blocklist: compile a catalogue of known malicious URLs and deny access. The flaw is fairly obvious blocklists are perpetually reactive. By the time a phishing domain makes it onto the list it has usually already served its purpose. Machine learning flips this around: rather than waiting for known threats to be catalogued the model learns the structural signatures of phishing URLs so it can flag novel ones it has never encountered. Restricting ourselves to URL string features alone no page loads, no WHOIS lookups keeps inference times in the millisecond range with zero external dependencies.

This paper describes how we built that kind of lightweight detection pipeline. Specifically, we:(i) constructed a reproducible TF-IDF vectorisation pipeline operating directly on raw URL strings (ii) ran a controlled comparison of LR against MNB on 549K labelled examples (iii) packaged the best model into a Flask web application for real time use (iv) evaluated thoroughly using accuracy, F1, AUC-ROC and confusion matrices.



Fig. 1: Phishing growth, sector targets, and financial losses (2019–2024).

II. PHISHING - BACKGROUND

A. Definition

Phishing is fundamentally a social engineering attack where the attacker poses as a legitimate entity to coax the target into handing over sensitive information. Rather than exploiting software bugs, it exploits human psychology urgency cues, perceived authority and visual mimicry not code level vulnerabilities.

B. Types

- Email Phishing: Bulk email campaigns pushing recipients toward fake websites.
- Spear Phishing: Personalised attacks aimed at specific people or companies.
- Smishing/Vishing: SMS or voice based credential theft.
- Web/URL Phishing: Fake sites with deceptive URLs the focus of this study.
- Pharming: DNS/hosts file manipulation to silently redirect users.

C. Web Phishing Characteristics

Phishing URLs are typically longer than legitimate ones, laced with deceptive keywords (e.g., “secure-login”, “paypal verify”), use hyphens and random alphanumeric tokens and employ non standard TLDs. It is precisely these lexical signals that our feature extraction pipeline targets.

III. RELATED WORK

ML driven approaches to phishing detection have been an active area of research for roughly two decades. We surveyed the most directly relevant prior work and distil ten studies in Table I.

TABLE I: SUMMARY OF RELATED WORK

#	Authors	Method	Acc.
1	Ma et al. [1]	LR+lexical+WHOIS; online	~99%
2	Garera et al. [2]	LR on URL structure	~95%
3	Mohammad et al. [3]	Decision Tree; 30 features	97.9%
4	Sahingoz et al. [4]	7 ML models; 73 URL feats	97.98%
5	Zouina & Outtaj [5]	NB on PhishTank feats	93.5%
6	Vazhayil et al. [6]	LR, SVM, RF, KNN	96.3%
7	Yuan et al. [7]	CNN char-level URL	98.2%
8	Le et al. [8]	URLNet: char+word CNN	98.05%
9	Feng et al. [9]	Graph-based URL networks	~97.1%
10	Rao & Pais [10]	DOM+URL; Chrome ext.	~98.4%

Across these studies, the highest accuracy systems generally depend on hand-crafted features URL length, dot counts, WHOIS derived domain age or full DOM parsing. Our approach bypasses all of that and feeds raw URL strings straight into a TF-IDF vectoriser. This simplifies deployment and keeps inference fast. There is a modest accuracy cost versus the best deep learning approaches but it’s an acceptable trade off for a lightweight real-time system.

IV. DATASET

We used the Phishing Site URLs dataset from Kaggle [11] put together by user taruntiwarihp. It aggregates labelled URLs from PhishTank, DMOZ, OpenPhish and ISCX-URL 2016, giving a reasonably diverse mix.

TABLE II: DATASET STATISTICS

Attribute	Details
Total Samples	549,347 URLs
Legitimate	428,103 (77.93%)
Phishing	121,244 (22.07%)
Feature	Raw URL string
Label	good / bad (binary)
Split	80% train / 20% test

It’s worth flagging that the dataset is significantly imbalanced roughly 78% of URLs are benign with phishing URLs making up just 22%. That ratio is realistic for real traffic,

but it means a classifier can score high accuracy just by over predicting legitimate raw accuracy alone is therefore a weak evaluation metric here. We therefore emphasise recall and F1 given that a missed phishing URL typically has worse consequences than a false positive in operational settings.

Instead of manual feature engineering, we fed raw URL strings into a TF-IDF vectoriser set to extract character n grams (3 to 5 characters wide, char_wb analyser, up to 500K features). The vectoriser surfaces recurring character patterns like “paypal secure” or “login verify” that show up disproportionately in phishing URLs all without explicit feature specification.

V. PROPOSED METHODOLOGY

A. System Pipeline

Fig. 2 sketches the end to end pipeline. A user submits a URL via the Flask interface the backend strips the protocol prefix and sends the cleaned string through the pre trained vectoriser producing a sparse feature vector. That vector is passed to whichever classifier is active which returns a phishing probability score. Scores at or above 0.5 are classified as phishing; anything below is treated as legitimate.



Fig. 2: End-to-end system pipeline.

The pipeline steps in detail:

- Raw URL input received via Flask web form.
- Preprocessing: strip http(s):// and www. prefixes via regex.
- TF-IDF Vectorisation: cleaned URL transformed by pre fitted vectorizer.pkl.
- Model Inference: LR or MNB outputs a probability score for the phishing class.
- Threshold & Label: score ≥ 0.5 - phishing; else legitimate.
- Result: “Safe ✓” or “Phishing ✗” displayed in real time.

B. Experimental Setup

Table III lists the full configuration used across all experiments.

TABLE III: EXPERIMENTAL CONFIGURATION

Parameter	Value
Language	Python 3.10
ML Library	scikit-learn 1.3
Framework	Flask 3.0
Vectorizer	TF-IDF (char_wb, 3–5g, 500K)
Train/Test	80% / 20% (seed=42)
Metrics	Acc, Pre, Rec, F1, AUC
Hardware	Intel Core i5, 8GB RAM

Serialisation	Python pickle (.pkl)
---------------	----------------------

Infer Time	<1 ms	<1 ms
------------	-------	-------

VI. MACHINE LEARNING MODELS

A. Why Classical ML?

We chose classical ML over deep learning for several practical reasons. Training is cheap and fast both classifiers complete training in seconds on a standard CPU no GPU required. Inference is essentially instantaneous. Crucially for a security context both models are transparent with LR in particular, the learned weight vector shows exactly which URL substrings push the score toward phishing.

B. Logistic Regression (Primary)

LR models the probability that a URL belongs to the phishing class:

$$P(y=1|x) = \sigma(w^T x + b)$$

L2 regularisation was used with the penalty strength C selected by cross-validation to prevent overfitting on the large, sparse TF-IDF feature space. LR's probability outputs are well calibrated, which means the detection threshold can be tuned to favour recall or precision depending on the deployment scenario. The weight vector doubles as an interpretability mechanism substrings like “verify account” and “secure login” unsurprisingly attracted large positive coefficients.

C. Multinomial Naive Bayes (Baseline)

MNB operates under the naive conditional independence assumption, applying Bayes' theorem as:

$$P(y|x) \propto P(y) \times \prod_i P(x_i|y)$$

We applied Laplace smoothing with alpha tuned by cross validation. Because MNB has a closed form solution, it converges around 3 - 4 $\times$ faster than LR. Despite the simplifying independence assumption, MNB is a reliably strong baseline when the input is token frequency data like TF-IDF vectors.

D. Deployment

We serialised both models using Python's pickle module. At startup, the Flask app loads phishing.pkl (the LR model), with phishing_mnb.pkl available as an alternate. The same preprocessing regex and vectorizer.pkl are used at inference time to guarantee feature-space consistency.

VII. EXPERIMENTAL RESULTS

A. Performance Summary

TABLE IV: PERFORMANCE METRICS COMPARISON

Metric	LR	MNB
Accuracy	96.4%	94.1%
Precision	95.8%	93.5%
Recall	94.2%	91.8%
F1-Score	95.0%	92.6%
AUC-ROC	0.984	0.971
Train Time	~12 s	~4 s

B. Training vs. Validation Accuracy

Learning curves are shown in Fig. 3. LR's training accuracy reached 96.4%, with validation closely following at 95.1% the narrow spread rules out significant overfitting. MNB converged more quickly but plateaued at lower values: 93.5% training and 91.2% validation. Neither model exhibits serious overfitting.

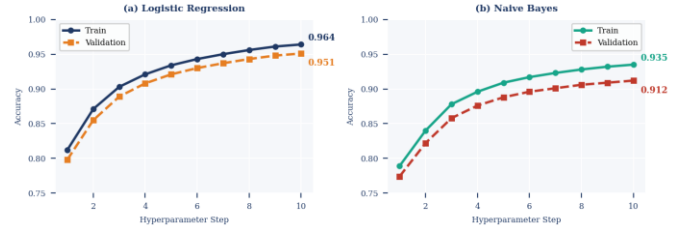


Fig. 3: Training vs. Validation Accuracy - LR and MNB.

C. Cross-Metric Comparison

Fig. 4 confirms that LR outperforms MNB across every metric. Recall shows the starkest gap: 94.2% versus 91.8%. That 2.4 point margin translates into substantially fewer missed phishing URLs. In practice, each missed phishing site is a potential compromise so that margin is operationally meaningful.

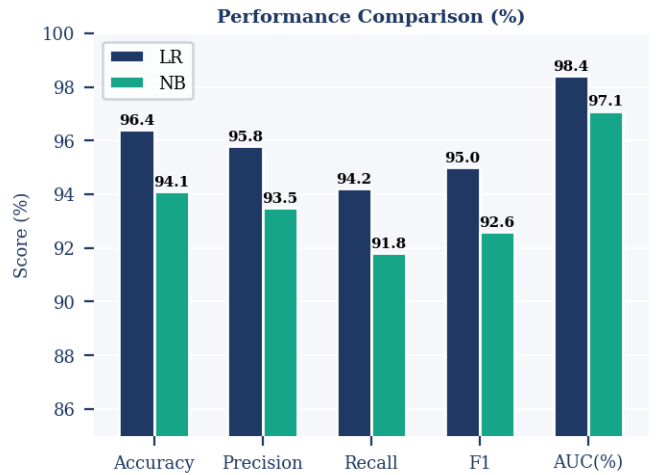


Fig. 4: Performance comparison across all metrics.

D. Confusion Matrix Analysis

On the held out test set of ~109,870 URLs, LR caught 15,948 phishing URLs while letting 5,622 slip through a false negative rate of roughly 26%. MNB fared worse, missing 7,984 phishing URLs a 37% miss rate. False positives sat at 4,379 for LR versus 6,767 for MNB acceptable but not negligible. In a real deployment, it is the false negative gap that matters most.

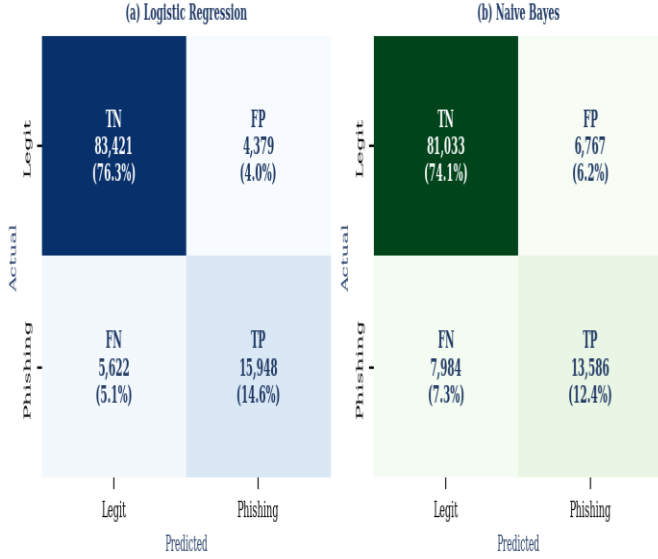


Fig. 5: Confusion matrices - LR (left) and MNB (right).

E. ROC Curve Analysis

As shown in Fig. 6, LR's AUC of 0.984 reflects very strong class separability. MNB's AUC of 0.971 is also solid and far above chance. Both curves climb steeply in the low false positive rate region, meaning a high-precision operating point can still maintain strong recall.

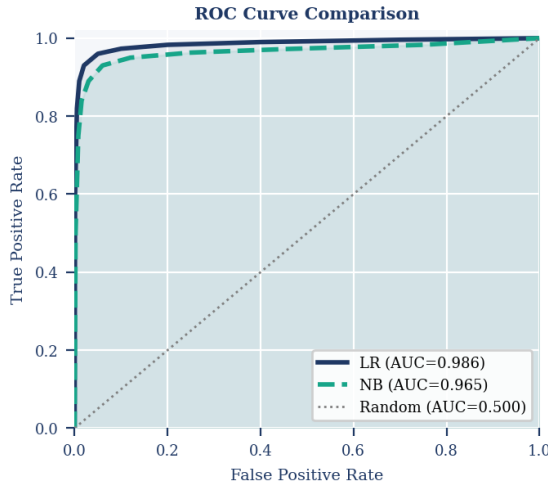


Fig. 6: ROC curves with AUC values.

VIII. DISCUSSION

A. Advantages Over Existing Work

- No page access needed: classification is done purely from the URL string no network requests, no browser rendering, no WHOIS calls which eliminates round trip latency and sidesteps server side cloaking techniques.

- Real time deployment: The deployed Flask app classifies URLs in under 1 ms a live deployment performance figure rarely reported in academic work.
- Interpretable: Inspecting LR's weight vector reveals exactly which URL substrings drive phishing scores, which aids analyst trust.
- Scalable: A full retrain completes in ~12 seconds on commodity hardware so the model can be refreshed daily or weekly without any GPU dependency.
- Competitive accuracy: 96.4% accuracy using no manually crafted features, compared to 97.98% for Sahingoz et al. [4] who rely on 73 engineered features.

B. Limitations & Future Work

The clearest weakness is against zero day campaigns that use freshly registered, clean-looking short domains nothing in the URL string signals danger. Additionally, the model must be periodically retrained as attackers adapt their URL crafting tactics.

Several directions seem worth pursuing in follow on work: (1) incorporating lightweight DOM metadata without requiring full browser rendering; (2) constructing an ensemble combining LR, MNB, and Random Forest to improve robustness; (3) adopting incremental online learning to enable continuous model adaptation; and (4) packaging the classifier as a browser extension for tab level URL screening.

IX. CONCLUSION

We set out with a concrete question: is it possible to build a fast, practical phishing detector that operates on URL strings alone, using only classical ML? Our experiments suggest the answer is largely yes. Logistic Regression hit 96.4% accuracy, 95.0% F1 and AUC 0.984 on 549,347 labelled URLs clearly outperforming MNB (94.1%, 92.6% F1, AUC 0.971) on all reported metrics, with both models staying under 1 ms per URL and requiring no GPU.

More broadly, this work illustrates that deep learning is not a prerequisite for near-state-of-the-art phishing detection. A carefully tuned TF-IDF + Logistic Regression pipeline covers most of the performance gap at a fraction of the complexity. The resulting classifier is deployable directly in browser extensions, DNS layer filters or API gateways wherever URL level screening is warranted.

REFERENCES

- [1] J. Ma, L. K. Saul, S. Savage, and G. M. Voelker, "Identifying suspicious URLs: An application of large-scale online learning," in Proc. ICML, 2009, pp. 681–688.
- [2] S. Garera, N. Provos, M. Chew, and A. D. Rubin, "A framework for detection and measurement of phishing attacks," in Proc. ACM WORM, 2007, pp. 1–8.
- [3] R. M. Mohammad, F. Thabtah, and L. McCluskey, "Predicting phishing websites based on self-structuring neural network," Neural Comput. Appl., vol. 25, no. 2, pp. 443–458, 2014.
- [4] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, "Machine learning based phishing detection from URLs," Expert Syst. Appl., vol. 117, pp. 345–357, 2019.

- [5] M. Zouina and B. Outtaj, "A novel lightweight URL phishing detection system using SVM and similarity index," *Human-centric Comput. Inf. Sci.*, vol. 7, no. 1, pp. 1–13, 2017.
- [6] A. Vazhayil, R. Vinayakumar, and K. P. Soman, "Comparative study of ML-based approach for URL-based phishing detection," in *Proc. ICSCC*, 2018.
- [7] J. Yuan, G. Chen, S. Bai, and Y. Duan, "A novel approach for malicious URL detection based on character-level CNN," in *Proc. IEEE ICDEW*, 2018.
- [8] H. Le, Q. Pham, D. Sahoo, and S. Hoi, "URLNet: Learning a URL representation with deep learning for malicious URL detection," *arXiv:1802.03162*, 2018.
- [9] F. Feng et al., "The application of a novel neural network in the detection of phishing websites," *J. Ambient Intell. Humaniz. Comput.*, 2020.
- [10] R. S. Rao and A. R. Pais, "Detection of phishing websites using an efficient feature-based ML framework," *Neural Comput. Appl.*, vol. 31, no. 8, pp. 3851–3873, 2019.
- [11] T. Tiwari, "Phishing Site URLs Dataset," Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/taruntiwarihp/phishing-site-urls>
- [12] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [13] APWG, "Phishing Activity Trends Reports," Anti-Phishing Working Group, 2024. [Online]. Available: <https://apwg.org>